

# HW04 - AI Automation Testing Report

Student: Nguyen Phan Hung Linh Student ID: 23127081 Repository:  
<https://github.com/linhnph05/hw04---Automation-testing> Date: 2026-08-09

## 1. Scope and feature selection

I automated these three web features:

- Pool A - FR-03: Forgot password and reset password.
- Pool B - FR-09: Discount coupons.
- Pool C - FR-12: Admin access control.

The local eshop-sut directory was treated as read-only. Tests and evidence are stored in the homework repository, so the suite does not hide defects by changing the SUT.

## 2. Method

I used a staged AI-first workflow. I first asked another Codex agent to inspect the requirements and source. Next, I asked it to design exactly 12 cases per feature. Only after reviewing those cases did I request a Playwright draft for one feature at a time. I converted the drafts from CommonJS to the repository's ES module format, corrected weak assumptions, ran a Chromium baseline, asked the agent to critique its own drafts, and then made a final human-review pass.

The course notes recommend data-driven tests, role-based locators, web-first assertions, isolated state, API setup, and multi-browser execution. The final suite follows these points:

- Data is loaded from three external JSON files under `test-data/`.
- UI tests use Playwright role, placeholder, and form locators.
- API setup is used for fixture users and security checks.
- No fixed sleeps are used in the tests.
- Unique fixture IDs and cleanup prevent tests from depending on execution order.
- Each feature runs separately in Chromium, Firefox, and WebKit.

## 3. Automated cases and assertion patterns

Each JSON file contains 12 rows, giving 36 test cases. Across three browser projects, this produces 108 executions.

All 36 cases are also documented with their type, layer, preconditions, input, actions, and expected result in `TEST_CASES.md` (`TEST_CASES.md`). This is the human-readable text version of the test design; the JSON files remain the data sources used by Playwright.

The suite uses more than three assertion patterns, including:

1. Visibility and count: `toBeVisible()` and `toHaveCount()`.
2. Exact values and status codes: `toBe()`.
3. Text and regular expressions: `toMatch()` and `toContainText()`.
4. Navigation: `toHaveURL()`.
5. Object structure: `toMatchObject()`.
6. Negative state checks: `not().toEqual()` and verification that unauthorized fixtures were not stored.

## 4. Execution results

| Feature | Chromium            | Firefox | WebKit  | Total passed | Total failed |
|---------|---------------------|---------|---------|--------------|--------------|
| FR-03   | 7 passed / 5 failed | 7 / 5   | 7 / 5   | 21           | 15           |
| FR-09   | 7 passed / 5 failed | 7 / 5   | 7 / 5   | 21           | 15           |
| FR-12   | 5 passed / 7 failed | 5 / 7   | 5 / 7   | 15           | 21           |
| Total   | 19 / 17             | 19 / 17 | 19 / 17 | 57           | 51           |

The failures are consistent across all three engines. This consistency and the requirement-based assertions indicate SUT defects rather than browser-specific flakiness. Each of the nine HTML reports was opened with Chromium and checked for a visible Run by: 23127081 title and ISO timestamp.

## 5. Human review of AI-generated drafts

| AI draft problem                                          | Human correction                                                                      | Why the AI missed it                                                                 |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Used CommonJS require in an ESM project                   | Converted all drafts to import and file-based JSON parsing                            | The model followed common Playwright examples without checking project configuration |
| FR-03 TC-01 mixed OTP length with step indicators         | Kept OTP format as the single primary assertion; TC-02 owns navigation controls       | The model tried to reduce repeated setup and combined requirements                   |
| Relied on alert wording for reset results                 | Accepted dialogs without using their exact text and verified URL/login state          | The model copied the current UI instead of testing the stable business result        |
| Guest coupon lookup matched the permanent login link      | Replaced the broad text poll with a direct login-URL expectation                      | The locator was not scoped to the coupon result area                                 |
| Pretended a deleted coupon represented an inactive coupon | Replaced it with a real empty-code case and documented the inactive-fixture gap       | No public API can create <code>is_active = 0</code>                                  |
| Sent <code>user_id</code> on normal coupon calls          | Normal calls now rely on JWT identity; only the spoofing case sends another ID        | The model followed the buggy client contract too closely                             |
| Mixed per-user isolation with broken percent arithmetic   | TC-11 now asserts successful isolation only; TC-01 owns exact percent math            | One case had two independent failure reasons                                         |
| FR-12 mutation tests checked only HTTP status             | Added state checks proving unauthorized products, categories, or coupons were created | The first draft focused on the access response but not its side effect               |
| Fixture names used only <code>Date.now()</code>           | Added counters to prevent same-millisecond collisions                                 | The model underestimated repeated setup speed with one worker                        |

## 6. Defects found

Eight root-cause issues were created on GitHub. They cover all repeated assertion failures without duplicating one issue for every browser:

- FR-03: four-digit OTP; missing step/navigation/confirmation controls; valid password rejected.
- FR-09: incorrect percent math; wrong equality boundary; missing authentication and trusted body user ID.
- FR-12: missing admin-role checks; unauthenticated product write APIs.

Full mappings, evidence, and issue links are in `BUG_REPORT.md` (`BUG_REPORT.md`).

## 7. Automation gap

The exact inactive-coupon branch (`is_active = 0`) could not be isolated through the available UI or API because the SUT exposes no supported way to create or deactivate that fixture. I did not modify SQLite directly because that would couple the web automation to SUT internals. The suite still tests nonexistent, empty, expired, below-minimum, equality, usage-limit, guest, and identity-spoofing conditions. A future test-fixture endpoint or seed option should add the inactive case.

## 8. Report evidence

- HTML reports: `reports/html/<feature>-<browser>/index.html`.
- Visual summaries: `evidence/*-chromium-report.png`.
- Every failure retains a Playwright trace and screenshot during execution; the committed HTML reports contain the final run details.

## 9. Git history limitation

There are more than eight meaningful commits that modify `.spec.js` files. However, those HW04 test commits were made on 2026-08-09 rather than across four days. I did not falsify Git author dates. The repository has older history on another date, but it does not satisfy the rule that the qualifying test-script commits themselves span four days.

## 10. Conclusion

The final suite is data-driven, uses multiple assertion styles, runs on three browser engines, and produces nine attributable reports. The stable cross-browser failures found important correctness and authorization defects. AI accelerated source analysis and draft generation, but human review was necessary to correct module format, selectors, fixture assumptions, assertion scope, and cleanup.